

HelixPlay

HelixPlay

将任何配备 **GPU** 的设备转化为云游戏终端

概要

HelixPlay 是一款可自行托管的云游戏平台，能将任何配备 GPU 的设备转变为远程串流主机，为桌面端、移动端、电视及浏览器客户端提供主机级游戏体验。该平台以 Go 为核心，采用单一代码库架构，包含 46 个子模块，并支持合作伙伴进行白标定制。

简介

HelixPlay 是一款可自行托管、开放且支持白标定制的云游戏平台。它能够将任何配备 GPU 的设备转化为远程串流主机，通过 WebRTC/QUIC 技术，为桌面端、移动端、电视及浏览器客户端提供主机级游戏体验，核心基于 Go，客户端技术栈则采用 Wails/Flutter/Angular。

详细介绍

HelixPlay 是一款云游戏平台，采用以 Go 为核心的单一代码库架构，包含 46 个 Git 子模块。它能够将用户已有的游戏 PC 转化为串流主机，为桌面端、移动端、电视及浏览器客户端提供主机级游戏体验——完全自行托管、开放源代码，并支持合作伙伴进行白标定制。其核心理念直白明了：硬件归你，服务归你，品牌归你，无需第三方云服务介入。

该平台的设计核心在于三端客户端的融合架构——这一大胆的技术抉择在其他方面均带来回报。Wails 桌面应用、Flutter 移动/电视应用以及 Angular 网页客户端均基于同一个 Go 核心构建，该核心编译为 WASM 以支持浏览器运行，因此行为逻辑仅需编写一次，即可在所有平台共享，而非分叉三套代码。其底层则是实时媒体传输链路：采集 → 编码 → 打包 → 传输 → 解码 → 渲染，与平台原生采集技术（DXGI / ScreenCaptureKit / PipeWire）及硬件编码器（NVENC / QSV / AMF / VideoToolbox）深度集成，由 GPU 承担主要计算负载。传输层则采用 WebRTC（Pion v4）、QUIC（quic-go）及自定义 UDP 数据报协议，优先选择低延迟而非便捷性。后端

核心负责会话、租户、目录及身份验证管理；主机代理则在边缘端处理采集、编码及传输；mDNS/会合机制将设备发现过程无缝连接，客户端无需手动配置即可找到主机。

HelixPlay 从架构设计之初即面向白标 SaaS 服务，支持按租户定制主题、目录过滤、OAuth2 认证及计费功能，合作伙伴可基于此构建完全自有品牌的服务，而非简单的换肤。此外，平台完全基于容器化设计：所有服务、数据库、构建、测试及扫描均在容器内运行，确保整个平台的部署与验证过程可复现。与 Helix 家族的其他成员一样，该平台遵循“反虚假承诺”原则，即测试通过即代表真实、可供终端用户使用的功能，而非仅仅是模拟测试通过。

为何开发此平台

商业云游戏服务封闭、集中且依赖租赁模式。HelixPlay 的诞生，正是为了让任何拥有 GPU 设备的用户都能自行搭建串流主机——开放、自托管且支持白标定制，无需依赖第三方服务。

为何这是变革性的创新

它将商业服务中长期割裂的三大要素融为一体：由你掌控硬件的自托管、驱动三套客户端栈的单一Go核心（确保功能同步落地），以及白牌多租户架构。这意味着合作伙伴能在自有GPU上部署完全定制化的云游戏服务——掌控用户体验、用户关系与商业模式——而非依赖他人云端资源，受制于其限制。

创新之处

- 三栈客户端融合：Wails、Flutter与Angular均基于同一Go核心（浏览器端为WASM），桌面端、移动端、电视端及Web端共享单一实现，避免三套独立代码库的分化。
- 可自托管的白牌**SaaS**：内置每租户主题定制、内容过滤、OAuth2认证及计费系统，平台以可品牌化产品形式交付，而非仅供演示。
- 现代低延迟传输：WebRTC（Pion）、QUIC及定制UDP协议，结合按平台优选的硬件编码器（NVENC/QSV/AMF/VideoToolbox），以响应速度为优先，而非便捷性。
- **46**模块解耦架构：所有组件均采用容器化设计，每个服务、数据库、构建、测试及扫描流程均独立运行于容器中。

最大技术挑战及解决方案

- 异构硬件下的低延迟流式传输。各操作系统与GPU对屏幕捕获与编码的实现方式不一，且延迟容忍度极低。解决方案：平台感知的捕获/编码路径——DXGI/ScreenCaptureKit/PipeWire等接口分别对接NVENC/

QSV/AMF/VideoToolbox, 通过WebRTC/QUIC/UDP传输, 确保每台设备均采用其最快的本地像素处理路径。

- 桌面端、移动端、电视端及**Web端**的单一产品形态。解决方案: 三栈客户端 (Wails、Flutter、Angular) 共享同一Go核心 (浏览器端编译为WASM), 一次修复或功能更新即可同步至所有四个平台, 无需四次移植。
- 多租户白牌运营。解决方案: 将每租户主题定制、内容过滤、OAuth2认证及计费功能直接集成至核心后端, 使租户隔离与品牌化成为平台基础能力, 而非针对每个客户的定制分支。

技术栈

- **Go (1.26.2主版本/1.25+子模块)**: 共享核心后端与主机代理, 单一语言可编译为原生二进制文件及WASM, 实现单核多客户端架构。
- **Wails v2**: 桌面端客户端, 将Go核心与嵌入式WebView绑定, 桌面应用直接复用核心逻辑, 无需重新实现。
- **Flutter 3.29+**: 移动端/电视端客户端, 通过FFI调用Go核心, 在手机与电视上实现原生UI, 无需额外后端支持。
- **Angular 17+**: Web端客户端, 运行同一Go核心 (编译为WASM), 使浏览器成为一等公民, 而非简化版本。
- **WebRTC/Pion v4、QUIC/quic-go、定制UDP**: 三种实时传输协议, 平台可根据网络与客户端条件选择最低延迟路径。
- **硬件编码器 (NVENC/QSV/AMF/VideoToolbox) 与平台捕获接口 (DXGI/ScreenCaptureKit/PipeWire)**: GPU加速的捕获与编码路径, 按平台优选, 确保编码不成为CPU瓶颈。
- **容器化 (Docker/Podman)**: 所有服务、数据库、构建、测试及扫描流程均运行于容器中, 确保整个系统可复现部署与验证。
- **mDNS/会合机制**: 零配置主机发现, 客户端可自动定位局域网内的流媒体主机。

内容

状态与诚信说明

- **状态**: 测试版。自述文件中提及的延迟目标 (局域网 ≤ 30 毫秒 / 广域网 ≤ 50 毫秒, p999)、“主机级 / PS4 Pro 级”定位, 以及测试矩阵单元数量, 均为项目自行设定的设计目标, 未经独立基准测试, 仅供参考。
- **许可证**: 待定。经 GitHub 与 API 检测, 未发现 LICENSE 文件——未验证 / 未声明。

优先级别: Helix-主级。